

operation. If the calls to the `int` class's `__mul__()` and `WizCoin` class's `__rmul__()` both return `NotImplemented`, Python raises a `TypeError` exception.

Only `WizCoin` objects can be added to each other. This guarantees that the first `WizCoin` object's `__add__()` method will handle the operation, so we don't need to implement `__radd__()`. For example, in the expression `purse + tipJar`, the `__add__()` method for the `purse` object is called with `tipJar` passed for the other parameter. Because this call won't return `NotImplemented`, Python doesn't try to call the `tipJar` object's `__radd__()` method with `purse` as the other parameter.

[Table 17-2](#) contains a full listing of the available reflected dunder methods.

[Table 17-2](#): Reflected Numeric Dunder Methods

Dunder method	Operation	Operator or built-in function
<code>__radd__()</code>	Addition	<code>+</code>
<code>__rsub__()</code>	Subtraction	<code>-</code>
<code>__rmul__()</code>	Multiplication	<code>*</code>
<code>__rmatmul__()</code>	Matrix multiplication (new in Python 3.5)	<code>@</code>
<code>__rtruediv__()</code>	Division	<code>/</code>
<code>__rfloordiv__()</code>	Integer division	<code>//</code>
<code>__rmod__()</code>	Modulus	<code>%</code>
<code>__rdivmod__()</code>	Division and modulus	<code>divmod()</code>
<code>__rpow__()</code>	Exponentiation	<code>**</code> , <code>pow()</code>
<code>__rlshift__()</code>	Left shift	<code>>></code>
<code>__rrshift__()</code>	Right shift	<code><<</code>
<code>__rand__()</code>	Bitwise and	<code>&</code>
<code>__ror__()</code>	Bitwise or	<code> </code>
<code>__rxor__()</code>	Bitwise exclusive or	<code>^</code>

The full documentation for the reflected dunder methods is in the Python documentation at <https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>.

In-Place Augmented Assignment Dunder Methods